

Program:

```
import pandas as pd
from sklearn_crfsuite import CRF
from sklearn_crfsuite import metrics
# Sample training data: list of sentences with word-label pairs
train_data = [
    [('John', 'B-PER'), ('lives', 'O'), ('in', 'O'), ('New', 'B-LOC'), ('York', 'I-LOC')]
]
# Sample test data: list of sentences with word-label pairs
test_data = [
    [('Mary', 'B-PER'), ('is', 'O'), ('from', 'O'), ('Los', 'B-LOC'), ('Angeles', 'I-LOC')]
]

# Feature extraction function
def extract_features(sentence):
    return [{
        'word': word,
        'is_capitalized': word[0].upper() == word[0],
        'is_numeric': word.isdigit(),
        'is_upper': word.isupper(),
        'is_title': word.istitle(),
        'prev_word': sentence[i-1][0] if i > 0 else "",
        'next_word': sentence[i+1][0] if i < len(sentence)-1 else ""
    } for i, (word, _) in enumerate(sentence)]

# Preparing the training data
X_train = [extract_features(s) for s in train_data]
y_train = [[label for _, label in s] for s in train_data]

# Creating and training the CRF model
crf = CRF(algorithm='lbfgs', max_iterations=100)
crf.fit(X_train, y_train)

# Preparing the test data
X_test = extract_features(test_data[0]) # Testing with "Mary is from Los Angeles"
y_pred = crf.predict_single(X_test)

# Displaying predictions
print("Predicted labels for the test sentence:", y_pred)

# For evaluation, you could print the expected output
expected_labels = [label for _, label in test_data[0]]
print("Expected labels:", expected_labels)
```


Output:

Predicted labels for the test sentence: ['B-PER', 'O', 'O', 'B-LOC', 'I-LOC']

Expected labels: ['B-PER', 'O', 'O', 'B-LOC', 'I-LOC']